TECHNICAL TRAINING COURSE

Java API for

RESTful Services

JAX-RS 2.1 · javax.ws.rs.* · XML

Applications · Resources · Sub-resources
Providers · Filters · Scanning
Platform: Java 11 | Tomcat 9.0 | Jersey 2.41

Java 11 | Tomcat 9 | javax.* | XML

COURSE AGENDA

5 Sections | Applications, Resources, Sub-resources, Providers, Scanning

01 Applications

02 Resources — XML Producing

03 Sub-resources

04 Providers

05 Scanning & @ApplicationPath

The JAX-RS Application Class

USE: import javax.ws.rs.* | NOT: import jakarta.ws.rs.* | Tomcat 9 = javax.* namespace

```
import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;
import java.util.Set;

@ApplicationPath("/api")
public class ShopApplication extends Application {

    @Override
    public Set<Class<?>> getClasses() {
        return Set.of(
            ProductResource.class,
            OrderResource.class,
            NotFoundExceptionMapper.class,
            CorsFilter.class
        );
    }
}
```

`@ApplicationPath("/api")`

Sets the base URI — all @Path resources root here

`getClasses()`

Manual registration; returns a fresh instance per request

`Empty body`

Enables automatic classpath scanning for @Path and @Provider

`getSingletons()`

Pre-built shared instances (e.g. ContextResolver<JAXBContext>)

Resource Methods — @Produces(APPLICATION_XML)

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import com.example.model.Product;  // must be @XmlRootElement annotated

@Path("/products")
@Produces(MediaType.APPLICATION_XML)    // all methods return XML
@Consumes(MediaType.APPLICATION_XML)    // all methods accept XML bodies
public class ProductResource {

    @GET
    public List<Product> listAll() {
        return repo.findAll();
        // Jersey + JAXB marshals List<Product> → <products><product>...</product></products>
    }

    @GET @Path("{id}")
    public Response getById(@PathParam("id") String id) {
        Product p = repo.findById(id);
        if (p == null) return Response.status(404).build();
        return Response.ok(p).build();  // JAXB marshals Product → XML
    }

    @POST
    public Response create(Product product, @Context UriInfo uriInfo) {
        Product saved = repo.save(product);
        URI loc = uriInfo.getAbsolutePathBuilder().path(saved.getId()).build();
        return Response.created(loc).entity(saved).build();  // 201 + Location
    }
}
```

JAXB-Annotated Domain Model

```java
import javax.xml.bind.annotation.*;
import java.math.BigDecimal;

@XmlRootElement(name = "product")
@XmlAccessorType(XmlAccessType.FIELD)
@XmlType(propOrder = {
    "name","category","price","stockLevel"
})
public class Product {

    @XmlAttribute
    private String     id;

    @XmlElement(required = true)
    private String     name;

    @XmlElement(required = true)
    private String     category;

    @XmlElement(required = true)
    private BigDecimal price;

    @XmlElement
    private int        stockLevel;

    public Product() {}  // required by JAXB
}
```

```xml
<!-- GET /api/products/P001 response: -->

<?xml version="1.0" encoding="UTF-8"?>
<product id="P001">
  <n>Clean Code</n>
  <category>books</category>
  <price>39.99</price>
  <stockLevel>50</stockLevel>
</product>


<!-- POST /api/products request body: -->

<?xml version="1.0" encoding="UTF-8"?>
<product>
  <n>Effective Java</n>
  <category>books</category>
  <price>49.99</price>
  <stockLevel>30</stockLevel>
</product>

<!-- Response: 201 Created
     Location: /api/products/P007 -->
```

Content Negotiation — Accept Header

```
@GET @Path("{id}/summary")
@Produces({MediaType.APPLICATION_XML, MediaType.TEXT_PLAIN})
public Response getSummary(@PathParam("id") String id,
                           @Context Request request) {
    Product p = repo.findById(id);
    List<Variant> variants = Variant.mediaTypes(
        MediaType.APPLICATION_XML_TYPE, MediaType.TEXT_PLAIN_TYPE).build();
    Variant chosen = request.selectVariant(variants);
    if (MediaType.TEXT_PLAIN_TYPE.equals(chosen.getMediaType()))
        return Response.ok(p.getId() + " | " + p.getName(),
                           MediaType.TEXT_PLAIN).build();
    return Response.ok(p).build();  // JAXB → XML
}
```

`Accept: application/xml`	→ <product id='P001'>...</product>
`Accept: text/plain`	→ P001 | Clean Code | 39.99
`Accept: text/html`	→ 406 Not Acceptable

Parameter Injection Annotations

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@GET @Path("search")
@Produces(MediaType.APPLICATION_XML)
public Response search(
    @QueryParam("q")                  String query,
    @QueryParam("page")
        @DefaultValue("0")            int    page,
    @QueryParam("size")
        @DefaultValue("20")           int    size,
    @QueryParam("category")           String category,
    @HeaderParam("X-Request-ID")      String requestId,
    @Context UriInfo                  uriInfo) {

    List<Product> results = repo.find(
        query, category, page, size);
    return Response.ok(new ProductList(results)).build();
}
```

`@PathParam`
URI template variable {id}

`@QueryParam`
Query string ?key=value

`@HeaderParam`
HTTP request header

`@FormParam`
application/x-www-form-urlencoded field

`@DefaultValue`
Default when param is absent

`@Context`
UriInfo, HttpHeaders, Request, SecurityContext

`@BeanParam`
Aggregate params into a single bean class

Response Builder — javax.ws.rs.core.Response

```
import javax.ws.rs.core.Response;
import javax.ws.rs.core.MediaType;

// 200 OK — JAXB marshals Product → XML
return Response.ok(product).build();

// 201 Created + Location header + XML body
URI loc = uriInfo.getAbsolutePathBuilder().path(id).build();
return Response.created(loc).entity(saved).build();

// 204 No Content (DELETE)
return Response.noContent().build();

// 400 Bad Request — XML error bean as entity
return Response.status(Response.Status.BAD_REQUEST)
               .entity(new ApiError(400, "BAD_REQUEST", "Invalid SKU"))
               .type(MediaType.APPLICATION_XML)
               .build();

// 404 Not Found — XML error body
return Response.status(Response.Status.NOT_FOUND)
               .entity(new ApiError(404, "NOT_FOUND", "Product: " + id))
               .type(MediaType.APPLICATION_XML)
               .build();
```

Sub-resource Methods vs Sub-resource Locators

```
// SUB-RESOURCE METHOD
// has @Path AND @GET/@PUT etc.
@Path("/orders")
@Produces(APPLICATION_XML)
public class OrderResource {

    // GET /api/orders/ORD-001/status
    @GET
    @Path("{id}/status")
    @Produces(TEXT_PLAIN)
    public String getStatus(
            @PathParam("id") String id) {
        return service.findById(id)
                      .getStatus().name();
    }

    // PUT /api/orders/ORD-001/cancel
    @PUT
    @Path("{id}/cancel")
    public Response cancel(
            @PathParam("id") String id) {
        service.cancel(id);
        return Response.noContent().build();
    }
}
```

```
// SUB-RESOURCE LOCATOR
// has @Path, NO verb annotation
@Path("/stores")
public class StoreResource {

    @Path("{sid}/inventory")
    public InventoryResource locator(
            @PathParam("sid") String sid) {
        Store s = storeService.find(sid);
        if (s == null)
            throw new NotFoundException(
                "Store: " + sid);
        return new InventoryResource(s);
    }
}

// No @Path at class level
@Produces(APPLICATION_XML)
public class InventoryResource {
    @GET
    public List<InventoryItem> getAll()
        { return store.getInventory(); }

    @PUT @Path("{sku}")
    public InventoryItem update(...) {...}
}
```

SECTION 4 — PROVIDERS

Provider Types

`MessageBodyReader<T>`	Deserialise request body → Java	Custom XML parsing
`MessageBodyWriter<T>`	Serialise Java → response body	Custom XML, CSV, binary
`ExceptionMapper<E>`	Exception → HTTP Response	XML error bodies for 4xx/5xx
`ContainerRequestFilter`	Intercept incoming requests	Auth tokens, logging
`ContainerResponseFilter`	Intercept outgoing responses	CORS headers
`ContextResolver<T>`	Supply context to providers	Custom JAXBContext config

ExceptionMapper — XML Error Responses

```
// JAXB-annotated error bean
@XmlRootElement(name = "error")
@XmlAccessorType(XmlAccessType.FIELD)
public class ApiError {
    @XmlElement public int    status;
    @XmlElement public String code;
    @XmlElement public String message;
    public ApiError() {}
    public ApiError(int s, String c, String m)
        { status=s; code=c; message=m; }
}

// ExceptionMapper
import javax.ws.rs.NotFoundException;
import javax.ws.rs.ext.*;

@Provider
public class NotFoundExceptionMapper
        implements ExceptionMapper<NotFoundException> {
    @Override
    public Response toResponse(NotFoundException e) {
        return Response
            .status(404)
            .entity(new ApiError(404,
                "NOT_FOUND", e.getMessage()))
            .type(MediaType.APPLICATION_XML)
            .build();
    }
}
```

```
<!-- HTTP 404 response body: -->

<?xml version="1.0"?>
<error>
  <status>404</status>
  <code>NOT_FOUND</code>
  <message>
    Product not found: P999
  </message>
</error>

<!-- HTTP 401 response body: -->

<?xml version="1.0"?>
<error>
  <status>401</status>
  <code>UNAUTHORIZED</code>
  <message>
    Missing Bearer token
  </message>
</error>
```

ContainerRequestFilter + @NameBinding

```java
// 1. Define the binding annotation
import javax.ws.rs.NameBinding;

@NameBinding
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE,
         ElementType.METHOD})
public @interface Secured {}

// 2. Apply to the filter
import javax.annotation.Priority;
import javax.ws.rs.Priorities;
import javax.ws.rs.container.*;

@Provider @Secured
@Priority(Priorities.AUTHENTICATION)
public class BearerTokenFilter
    implements ContainerRequestFilter {
  public void filter(ContainerRequestContext c) {
    String auth = c.getHeaderString(
        HttpHeaders.AUTHORIZATION);
    if (auth == null ||
            !auth.startsWith("Bearer "))
      c.abortWith(Response.status(401)
        .entity(new ApiError(401,
          "UNAUTHORIZED","Missing token"))
        .type(APPLICATION_XML).build());
  }
}
```

```java
// 3. Apply selectively
@Path("/products")
public class ProductResource {

  @GET  // no filter — public
  public List<Product> listAll() {
      return repo.findAll();
  }

  @POST
  @Secured  // requires Bearer token
  public Response create(
          Product product,
          @Context UriInfo ui) {
      ...
  }

  @DELETE @Path("{id}")
  @Secured  // also secured
  public void delete(
          @PathParam("id") String id) {
      ...
  }
}
```

SECTION 5 — SCANNING

Scanning vs Manual Registration

Small app / single team Zero config; new classes auto-discovered	**Scanning (empty Application)**
Microservice, controlled Only expose intended endpoints	**Manual getClasses()**
API versioning (v1/v2) Isolate versions with separate getClasses()	**Multiple Applications**
Shared JAXBContext Expensive context created once	**getSingletons() + ContextResolver**
Production (performance) No classpath scan overhead at startup	**Manual registration**

COURSE SUMMARY

Key Takeaways

01 import javax.ws.rs.* — NOT jakarta.ws.rs.* on Tomcat 9 / Java 11

02 @Produces(APPLICATION_XML) + JAXB @XmlRootElement = automatic XML marshalling

03 @Consumes(APPLICATION_XML) unmarshals XML request bodies via JAXB

04 Error responses should also return XML — use a @XmlRootElement error bean

05 Sub-resource METHOD = @Path + verb. LOCATOR = @Path, no verb, returns object

06 ExceptionMapper + @Provider = structured XML errors for all 4xx/5xx responses

07 @NameBinding + @Priority controls which filters run on which endpoints